

You made this submission 10 days ago.

Score: 35.00 Status: **Accepted**

People who solved **Sorting: Comparator** attempted this next:

Simplified Chess Engine II

Can you take opponent's Queen before losing your own?

[Solve Challenge](#)

Submitted Code

Language: Java 15

[Open in editor](#)

NEED HELP?

[View discussions](#)[View editorial](#)[View top submissions](#)

✓ Test case 0

✓ Test case 1 

✓ Test case 2 

✓ Test case 3 

✓ Test case 4 

✓ Test case 5 

✓ Test case 6

Compiler Message

Success

Input (stdin)

```
1 5
2 amy 100
3 david 100
4 heraldo 50
5 aakansha 75
6 aleksa 150
```

Expected Output

```
1 aleksa 150
```

[Download](#)[Download](#)

You are given a string containing characters *A* and *B* only. Your task is to change it into a string such that there are no matching adjacent characters. To do this, you are allowed to delete zero or more characters in the string.

Your task is to find the minimum number of required deletions.

Example

s = *AABAAB*

Remove an *A* at positions **0** and **3** to make *s* = *ABAB* in **2** deletions.

Function Description

Complete the `alternatingCharacters` function in the editor below.

`alternatingCharacters` has the following parameter(s):

- string *s*: a string

Returns

- int: the minimum number of deletions required

Input Format

The first line contains an integer *q*, the number of queries.

The next *q* lines each contain a string *s* to analyze.

Constraints

- $1 \leq q \leq 10$
- $1 \leq \text{length of } s \leq 10^5$
- Each string *s* will consist only of characters *A* and *B*.

Sample Input

```
5
AAAA
BBBBB
ABABABAB
BABABA
AAABBB
```

Sample Output

```
3
1
0
4
1
```

Upload Code as File

☐ Test against custom input

Run Code

Submit

Problem Solving
★★

You have earned 20.00 points!
You are now 5 points away from the 3rd star for your problem solving badge.

95%

195

Congratulations

You solved this challenge. Would you like to challenge your friends? [Twitter](#) [LinkedIn](#)

Next Challenge

Test case 0

Test case 1

Test case 2

Test case 3

Test case 4

Test case 5

Test case 6

Compiler Message

Success

Input (stdin)

1 5
2 AAAA
3 BBBBB
4 ABABABAB
5 BABABA
6 AAABBB

Expected Output

We define subsequence as any subset of an array. We define a subarray as a contiguous subsequence in an array.

Given an array, find the maximum possible sum among:

1. all nonempty subarrays.
2. all nonempty subsequences.

Print the two values as space-separated integers on one line.

Note that empty subarrays/subsequences should not be considered.

Example

`arr = [-1, 2, 3, -4, 5, 10]`

The maximum subarray sum is comprised of elements at indices `[1 - 5]`. Their sum is `2 + 3 + -4 + 5 + 10 = 16`.

The maximum subsequence sum is comprised of elements at indices `[1, 2, 4, 5]` and their sum is

`2 + 3 + 5 + 10 = 20`.

Function Description

Complete the `maxSubarray` function in the editor below.

`maxSubarray` has the following parameter(s):

- `int arr[n]`: an array of integers

Returns

- `int[2]`: the maximum subarray and subsequence sums

Input Format

The first line of input contains a single integer `t`, the number of test cases.

The first line of each test case contains a single integer `n`.

The second line contains `n` space-separated integers `arr[i]` where $0 \leq i < n$.

Constraints

- $1 \leq t \leq 10$
- $1 \leq n \leq 10^5$
- $-10^4 \leq arr[i] \leq 10^4$

The subarray and subsequences you consider should have at least one element.

Sample Input 0

87
88

}

Line: 88 Co



Upload Code as File



Test against custom input

Run Code

Submit C



You have earned 50.00 points!

You are now 25 points away from the 3rd star for your problem solving badge.

75%

175/2

Congratulations

You solved this challenge. Would you like to challenge your friends? [Twitter](#) [LinkedIn](#)

Next Challenge



Test case 0



Test case 1



Test case 2



Test case 3



Test case 4

Compiler Message

Success

Input (stdin)

```
1 6
2 1
3 1
4 6
5 -1 -2 -3 -4 -5 -6
6 2
7 1 -2
```

Download

Two children, Lily and Ron, want to share a chocolate bar. Each of the squares has an integer on it.

Lily decides to share a contiguous segment of the bar selected such that:

- The length of the segment matches Ron's birth month, and,
- The sum of the integers on the squares is equal to his birth day.

Determine how many ways she can divide the chocolate.

Example

$s = [2, 2, 1, 3, 2]$

$d = 4$

$m = 2$

Lily wants to find segments summing to Ron's birth day, $d = 4$ with a length equalling his birth month, $m = 2$. In this case, there are two segments meeting her criteria: $[2, 2]$ and $[1, 3]$.

Function Description

Complete the birthday function in the editor below.

birthday has the following parameter(s):

- int $s[n]$: the numbers on each of the squares of chocolate
- int d : Ron's birth day
- int m : Ron's birth month

Returns

- int: the number of ways the bar can be divided

Input Format

The first line contains an integer n , the number of squares in the chocolate bar.

The second line contains n space-separated integers $s[i]$, the numbers on the chocolate squares where $0 \leq i < n$.

The third line contains two space-separated integers, d and m , Ron's birth day and his birth month.

Constraints

- $1 \leq n \leq 100$
- $1 < s[i] < 5$, where $(0 < i < n)$

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Problem Solving

★★

You have earned 10.00 points!

You are now 75 points away from the 3rd star for your problem solving badge.

25%

125/200

Congratulations

You solved this challenge. Would you like to challenge your friends?

Twitter

LinkedIn

Next Challenge

Test case 3

Test case 4

Test case 5

Test case 6

Test case 7

Test case 8

Test case 9

Compiler Message

Success

Input (stdin)

15

21 2 1 3 2

33 2

Expected Output

12

Download

Download

Accepted 69 / 69 testcases passed

kimidibindhu2 submitted at Aug 22, 2026 09:19

Analysis

Solution



Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.

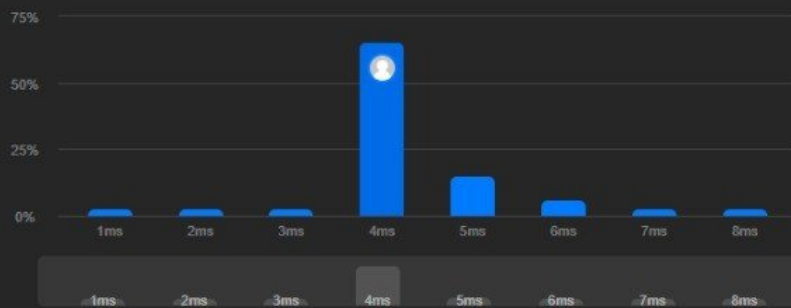


Runtime

4 ms | Beats 96.77%

Memory

43.75 MB | Beats 47.74%



Code | Java

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 class Solution {
5     public List<String> stringMatching(String[] words) {
6         List<String> result = new ArrayList<>();
7         int n = words.length;
8     }
```

View more

More challenges

Java Auto

```
6 List<String> result = new ArrayList<>();
7 int n = words.length;
8
9 for (int i = 0; i < n; i++) {
10     for (int j = 0; j < n; j++) {
11         // Do not compare a word with itself
12         if (i != j && words[j].contains(words[i])) {
13             result.add(words[i]);
14             break; // Found a match, move to the next word
15         }
16     }
17 }
18
19 return result;
20 }
21 }
```

Saved

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Case 3

Input

words =
["mass", "as", "hero", "superhero"]

Output

["as", "hero"]

Expected

["as", "hero"]

Contribute a testcase

Accepted 50 / 50 testcases passed

kimidibindhu2 submitted at Aug 22, 2026 09:15

Analysis

Solution



Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.



Runtime

0 ms | Beats 100.00%

Memory

43.61 MB | Beats 78.29%



Code | Java

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 class Solution {
5     public List<String> findAndReplacePattern(String[] words, String pattern)
6         List<String> result = new ArrayList<>();
7
8     for (String word : words) {
```

View more

More challenges

Java Auto

```
23 char w = word.charAt(i);
24
25 int pIndex = p - 'a' + 1; // 1-based indexing to distinguish from default 0
26 int wIndex = w - 'a' + 1;
27
28 if (pToW[p - 'a'] == 0 && wToP[w - 'a'] == 0) {
29     pToW[p - 'a'] = wIndex;
30     wToP[w - 'a'] = pIndex;
31 } else if (pToW[p - 'a'] != wIndex || wToP[w - 'a'] != pIndex) {
32     return false;
33 }
34
35 }
36
37 return true;
38 }
```

Saved

Testcase Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Input

```
words =
["abc","deq","mee","aqq","dkd","ccc"]
```

```
pattern =
"abb"
```

Output

["mee","aqq"]

Expected

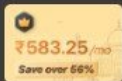
["mee","aqq"]

Accepted 1036 / 1036 testcases passed

kimidibindhu2 submitted at Aug 22, 2026 09:12

Analysis

Solution



Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.



Runtime

6 ms | Beats 80.77%

Memory

47.82 MB | Beats 24.73%



Code | Java

```
1 import java.util.Arrays;
2
3 class Solution {
4     public int lengthOfLongestSubstring(String s) {
5         int n = s.length();
6         int maxLength = 0;
7
8         // Array to store the last seen index of ASCII characters (initializ
```

View more

Java Auto

```
15
16 // If character was seen inside the current window, move left boundary
17 if (lastSeen[curr] >= left) {
18     left = lastSeen[curr] + 1;
19 }
20
21 // Update last seen index of current character
22 lastSeen[curr] = right;
23
24 // Calculate current window length
25 maxLength = Math.max(maxLength, right - left + 1);
26 }
27
28 return maxLength;
29
30 }
```

Saved

Testcase Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Case 3

Input

s =
"abcabcbb"

Output

3

Expected

3

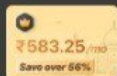
Contribute a testcase

Accepted 1096 / 1096 testcases passed

kimidibindhu2 submitted at Aug 22, 2026 09:07

Analysis

Solution



Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan. →

Runtime

1 ms | Beats 100.00%

Memory

43.59 MB | Beats 88.18%



Code | Java

```
1 class Solution {
2     public int myAtoi(String s) {
3         if (s == null || s.length() == 0) {
4             return 0;
5         }
6
7         int i = 0;
8         int n = s.length();
```

View more

More challenges

Java Auto

```
29     while (i < n && Character.isDigit(s.charAt(i))) {
30         int digit = s.charAt(i) - '0';
31
32         // Check for overflow before multiplying by 10
33         if (result > Integer.MAX_VALUE / 10 ||
34             (result == Integer.MAX_VALUE / 10 && digit > Integer.MAX_VALUE % 10)) {
35             return (sign == 1) ? Integer.MAX_VALUE : Integer.MIN_VALUE;
36         }
37
38         result = result * 10 + digit;
39         i++;
40     }
41
42     return result * sign;
43 }
44 }
```

Saved

Ln 4

Testcase Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Case 3

Case 4

Case 5

Input

```
s =
"42"
```

Output

42

Expected

42

Contribute a testcase

₹583.25
Save over 56%

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.

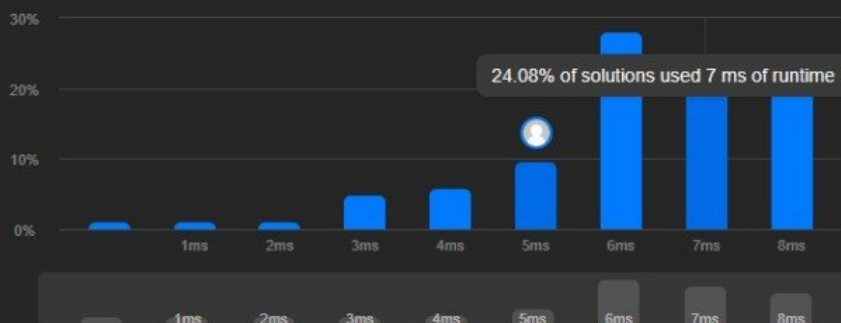


Runtime

5 ms | Beats 88.16%

Memory

51.14 MB | Beats 41.92%



Code | Java

```
1 class Solution {
2     public int maxSubarraySumCircular(int[] nums) {
3         int totalSum = 0;
4
5         int currentMax = 0;
6         int maxSum = nums[0];
7
8         int currentMin = 0;
```

View more

More challenges

- 3235. Check if the Rectangle Corner Is Reachable
- 3367. Maximize Sum of Weights after Edge Removals

Java Auto

```
21 // Modified Kadane's for Minimum Subarray Sum
22 currentMin += num;
23 minSum = Math.min(minSum, currentMin);
24 if (currentMin > 0) {
25     currentMin = 0;
26 }
27 }
28
29 // If all numbers are negative, return the maximum single element (maxSum)
30 if (maxSum < 0) {
31     return maxSum;
32 }
33
34 return Math.max(maxSum, totalSum - minSum);
35 }
36 }
```

Saved

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

nums =
[1,-2,3,-2]

Output

3

Expected

3

Contribute a testcase

Description | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 210 / 210 testcases passed

kimidibindhu2 submitted at Aug 22, 2026 08:56

Analysis

Solution

₹583.25/mo
Save over 68%

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.

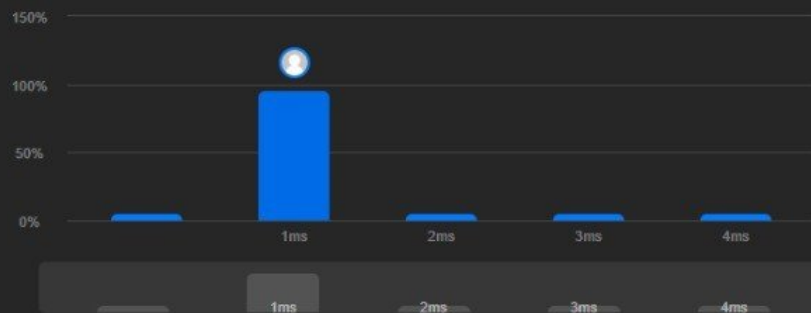


Runtime

1 ms | Beats 99.95%

Memory

77.12 MB | Beats 66.65%



Code | Java

```
1 class Solution {
2     public int maxSubArray(int[] nums) {
3         int maxSum = nums[0];
4         int currentSum = 0;
5
6         for (int num : nums) {
7             currentSum += num;
8             maxSum = Math.max(maxSum, currentSum);
9         }
10    }
```

View more

Code

Java | Auto

```
1 public class Solution {
2     public int maxSubArray(int[] nums) {
3         int maxSum = Integer.MIN_VALUE;
4         int currentSum = 0;
5
6         for (int num : nums) {
7             currentSum += num;
8             maxSum = Math.max(maxSum, currentSum);
9
10            // If current sum drops below zero, reset it
11            if (currentSum < 0) {
12                currentSum = 0;
13            }
14        }
15
16        return maxSum;
17    }
18 }
```

Saved

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Case 3

Input

nums =
[-2,1,-3,4,-1,2,1,-5,4]

Output

6

Expected

6

Contribute a testcase

More challenges